

Red Hat OpenShift Data Foundation 4.22

Red Hat OpenShift Data Foundation architecture

Overview of OpenShift Data Foundation architecture and the roles that the components and services perform.

Last Updated: 2026-09-04

Red Hat OpenShift Data Foundation 4.22 Red Hat OpenShift Data Foundation architecture

Overview of OpenShift Data Foundation architecture and the roles that the components and services perform.

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document provides an overview of the OpenShift Data Foundation architecture.

Table of Contents

PREFACE 4

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION 5

CHAPTER 1. INTRODUCTION TO OPENSHIFT DATA FOUNDATION 6

CHAPTER 2. AN OVERVIEW OF OPENSHIFT DATA FOUNDATION ARCHITECTURE 7

CHAPTER 3. OPENSHIFT DATA FOUNDATION OPERATORS 9
3.1. OPENSHIFT DATA FOUNDATION OPERATOR 9
3.2. OPENSHIFT CONTAINER STORAGE OPERATOR 10
3.2.1. Components 11
3.2.2. Design diagram 11
3.2.3. Responsibilities 11
3.2.4. Resources 12
3.2.5. Limitation 13
3.2.6. High availability 13
3.2.7. Relevant config files 13
3.2.8. Relevant log files 13
3.2.9. Lifecycle 13
3.3. ROOK-CEPH OPERATOR 14
3.3.1. Components 14
3.3.2. Design diagram 14
3.3.3. Responsibilities 15
3.3.4. Resources 16
3.3.5. Lifecycle 16
3.4. MCG OPERATOR 17
3.4.1. Components 17
3.4.2. Responsibilities and resources 17
3.4.3. High availability 19
3.4.4. Relevant log files 19
3.4.5. Lifecycle 19

CHAPTER 4. OPENSHIFT DATA FOUNDATION INSTALLATION OVERVIEW 20
4.1. INSTALLED OPERATORS 20
4.2. OPENSHIFT CONTAINER STORAGE INITIALIZATION 20
4.3. STORAGE CLUSTER CREATION 21
4.3.1. Internal mode storage cluster 21
4.3.1.1. Cluster Creation 22
4.3.1.2. NooBaa System creation 23
4.3.2. External mode storage cluster 24
4.3.2.1. Cluster Creation 25
4.3.2.2. NooBaa System creation 26
4.3.3. MCG Standalone StorageCluster 27
4.3.3.1. NooBaa System creation 27

CHAPTER 5. OPENSHIFT DATA FOUNDATION UPGRADE OVERVIEW 29
5.1. UPGRADE WORKFLOWS 29
5.2. CLUSTERSERVICEVERSION RECONCILIATION 29
5.3. OPERATOR RECONCILIATION 29

CHAPTER 6. NETWORKING PORTS 30
6.1. NETWORK PORTS FOR EXTERNAL MODE 30

6.1.1. Network ports required between OpenShift Container Platform and Ceph 30
6.2. NETWORK PORTS FOR INTERNAL MODE 30
6.2.1. Network ports required for internal mode deployment 30

CHAPTER 7. MULTUS ARCHITECTURE FOR OPENSHIFT DATA FOUNDATION 31
7.1. ASSUMPTIONS 31
7.2. EXAMPLE SCENARIO USED 31
7.3. HOW TO CHOOSE MULTUS ARCHITECTURE FOR AN OPENSHIFT DATA FOUNDATION DEPLOYMENT 32
7.3.1. Multus public network 32
7.3.2. Multus public and cluster networks 33
7.3.3. Multus cluster network 34
7.4. MULTUS NETWORKING REQUIREMENTS 34
7.4.1. General requirements 34
7.4.2. Routing requirements 35
7.4.3. Address space sizing 35
7.5. RECOMMENDATIONS FOR MULTUS NETWORKING 36
7.5.1. General recommendations for Multus networking 36
7.5.2. Address ranges 36
7.6. PLANNING HOST INTERFACE CONNECTIONS 36
7.7. MULTUS NETWORK CONFIGURATION 38
7.7.1. Public NetworkAttachmentDefinition 39
7.7.2. Cluster NetworkAttachmentDefinition 39
7.7.3. Node configuration for Multus public network 40
7.8. PREINSTALLATION CHECKLIST FOR MULTUS CONFIGURATION 41
7.9. VALIDATING AND DEBUGGING MULTUS CONFIGURATION 41
7.10. OPENSHIFT DATA FOUNDATION INSTALLATION AFTER VALIDATING MULTUS CONFIGURATION AND POST INSTALL CHECKS 42
7.11. ENABLING MULTUS SUPPORT ON AN EXISTING OPENSHIFT DATA FOUNDATION CLUSTER 43

APPENDIX A. MULTUS PREREQUISITE VALIDATION 46
A.1. TROUBLESHOOTING VALIDATION TOOL 47
A.2. HOW TO GET HELP 47

PREFACE

This document provides an overview of the OpenShift Data Foundation architecture.

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION

We appreciate your input on our documentation. Do let us know how we can make it better.

To give feedback, create a Jira ticket:

Procedure

1. Log in to the Jira.
2. Click **Create** in the top navigation bar
3. Enter a descriptive title in the Summary field.
4. Enter your suggestion for improvement in the Description field. Include links to the relevant parts of the documentation.
5. Select **Documentation** in the Components field.
6. Click Create at the bottom of the dialogue.

CHAPTER 1. INTRODUCTION TO OPENSHIFT DATA FOUNDATION

Red Hat OpenShift Data Foundation is a highly integrated collection of cloud storage and data services for Red Hat OpenShift Container Platform. It is available as part of the Red Hat OpenShift Container Platform Service Catalog, packaged as an operator to facilitate simple deployment and management.

Red Hat OpenShift Data Foundation services are primarily made available to applications by way of storage classes that represent the following components:

- Block storage devices, catering primarily to database workloads. Prime examples include Red Hat OpenShift Container Platform logging and monitoring, and PostgreSQL.



IMPORTANT

Block storage should be used for any worklaod only when it does not require sharing the data across multiple containers.

- Shared and distributed file system, catering primarily to software development, messaging, and data aggregation workloads. Examples include Jenkins build sources and artifacts, Wordpress uploaded content, Red Hat OpenShift Container Platform registry, and messaging using JBoss AMQ.
- Multicloud object storage, featuring a lightweight S3 API endpoint that can abstract the storage and retrieval of data from multiple cloud object stores.
- On premises object storage, featuring a robust S3 API endpoint that scales to tens of petabytes and billions of objects, primarily targeting data intensive applications. Examples include the storage and access of row, columnar, and semi-structured data with applications like Spark, Presto, Red Hat AMQ Streams (Kafka), and even machine learning frameworks like TensorFlow and Pytorch.



NOTE

Running PostgresSQL workload on CephFS persistent volume is not supported and it is recommended to use RADOS Block Device (RBD) volume. For more information, see the knowledgebase solution on ODF Database Workloads.

Red Hat OpenShift Data Foundation version 4.x integrates a collection of software projects, including:

- Ceph, providing block storage, a shared and distributed file system, and on-premises object storage
- Ceph CSI, to manage provisioning and lifecycle of persistent volumes and claims
- NooBaa, providing a Multicloud Object Gateway
- OpenShift Data Foundation, Rook-Ceph, and NooBaa operators to initialize and manage OpenShift Data Foundation services.

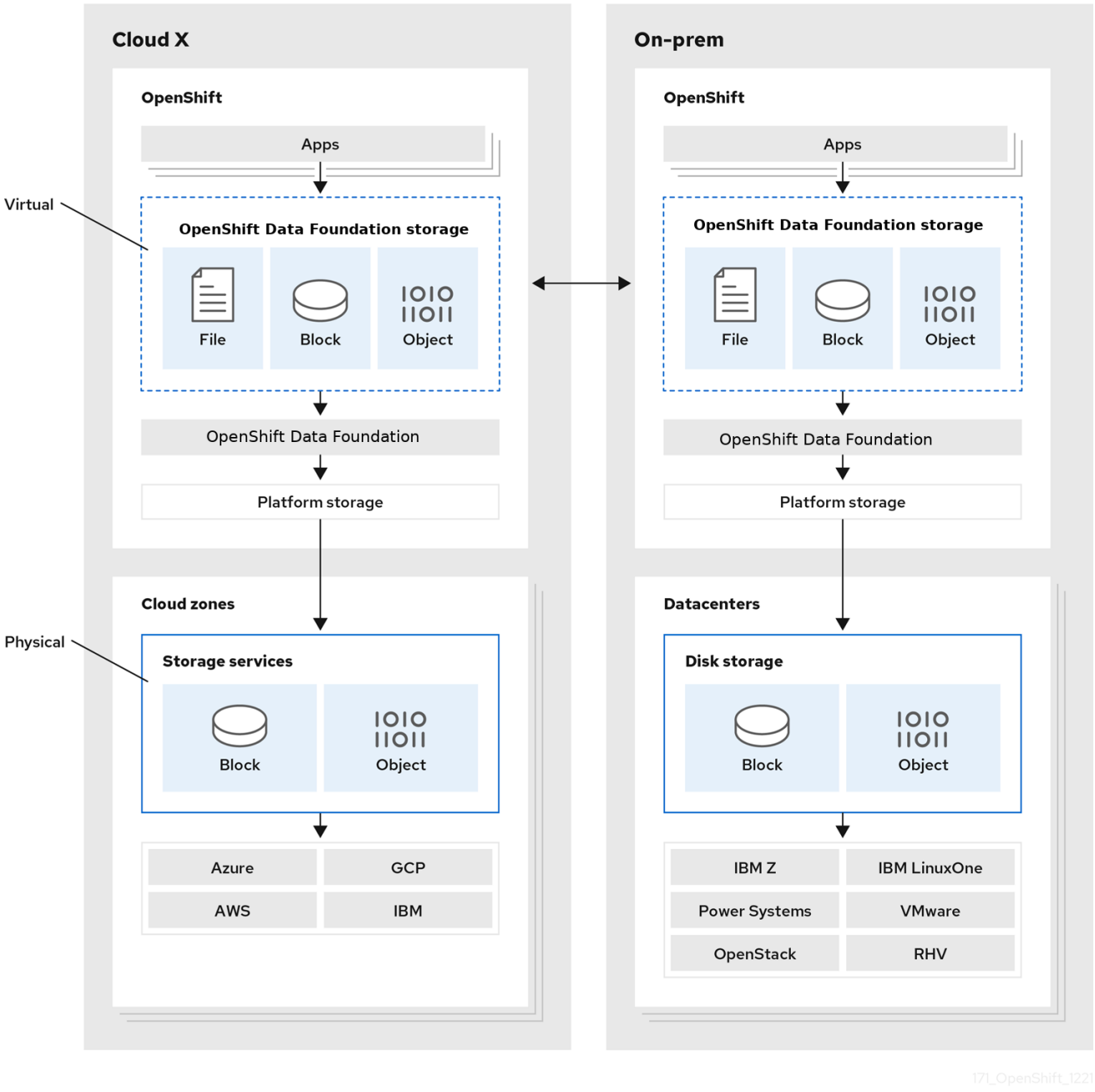
Additional resources

- ODF Database Workloads Must Not Use CephFS PVs/PVCs

CHAPTER 2. AN OVERVIEW OF OPENSHIFT DATA FOUNDATION ARCHITECTURE

Red Hat OpenShift Data Foundation provides services for, and can run internally from Red Hat OpenShift Container Platform.

Figure 2.1. Red Hat OpenShift Data Foundation architecture



Red Hat OpenShift Data Foundation supports deployment into Red Hat OpenShift Container Platform clusters deployed on Installer Provisioned Infrastructure or User Provisioned Infrastructure. For details about these two approaches, see the OpenShift Container Platform Installation process documentation. To know more about interoperability of components for the Red Hat OpenShift Data Foundation and Red Hat OpenShift Container Platform, see the interoperability matrix.

OpenShift Data Foundation uses failure domain configuration to determine how data replicas are distributed across the cluster. A failure domain represents a physical boundary, such as a host, rack, or availability zone within which replicas of the data are spread to ensure high availability. OpenShift Data

Foundation automatically identifies these failure domains based on the labels assigned to the storage nodes. For more information, see the knowledgebase article on OpenShift Data Foundation internal topology aware failure domain configuration.

In internal mode, OpenShift Data Foundation enables read-affinity by default to improve performance. This feature directs read operations to the nearest available OSD, reducing latency and minimizing cross-failure domain traffic. This behavior is especially valuable in public cloud environments, where it helps reduce expensive data transfers between Availability Zones. In stretched cluster deployments, read-affinity also prioritizes the local datacenter to ensure efficient, localized data access.

For information about the architecture and lifecycle of OpenShift Container Platform, see the OpenShift Container Platform architecture documentation.

Additional resources

- OpenShift Container Platform - Installation process
- Red Hat OpenShift Data Foundation interoperability matrix
- OpenShift Data Foundation internal topology aware failure domain configuration
- OpenShift Container Platform architecture

CHAPTER 3. OPENSHIFT DATA FOUNDATION OPERATORS

Red Hat OpenShift Data Foundation is comprised of the following three Operator Lifecycle Manager (OLM) operator bundles, deploying four operators which codify administrative tasks and custom resources so that task and resource characteristics can be easily automated:

- OpenShift Data Foundation
  - **odf-operator**
- OpenShift Container Storage
  - **ocs-operator**
  - **rook-ceph-operator**
- Multicloud Object Gateway
  - **mcg-operator**

Administrators define the desired end state of the cluster, and the OpenShift Data Foundation operators ensure the cluster is either in that state or approaching that state, with minimal administrator intervention.

3.1. OPENSHIFT DATA FOUNDATION OPERATOR

The **odf-operator** can be described as a "meta" operator for OpenShift Data Foundation, that is, an operator meant to influence other operators.

The **odf-operator** has the following primary functions:

- Enforces the configuration and versioning of the other operators that comprise OpenShift Data Foundation. It does this by using two primary mechanisms: operator dependencies and Subscription management.
  - The **odf-operator** bundle specifies dependencies on other OLM operators to make sure they are always installed at specific versions.
  - The operator itself manages the Subscriptions for all other operators to make sure the desired versions of those operators are available for installation by the OLM.
- Provides the OpenShift Data Foundation external plugin for the OpenShift Console.
- Provides an API to integrate storage solutions with the OpenShift Console.

Components

The **odf-operator** has a dependency on the **ocs-operator** package. It also manages the Subscription of the **mcg-operator**. In addition, the **odf-operator** bundle defines a second Deployment for the OpenShift Data Foundation external plugin for the OpenShift Console. This defines an **nginx**-based Pod that serves the necessary files to register and integrate OpenShift Data Foundation dashboards directly into the OpenShift Container Platform Console.

Design diagram

This diagram illustrates how **odf-operator** is integrated with the OpenShift Container Platform.

OpenShift Data Foundation Operator image::images/ocs-arch-odf-operator.png[OpenShift Container Storage Operator]

Resources

The **odf-operator** creates the Operator Lifecycle Manager Resources CR, which creates a **Subscription** for the operator.

Limitation

The **odf-operator** does not provide any data storage or services itself. It exists as an integration and management layer for other storage systems.

High availability

High availability is not a primary requirement for the **odf-operator** Pod similar to most of the other operators. In general, there are no operations that require or benefit from process distribution. OpenShift Container Platform quickly spins up a replacement Pod whenever the current Pod becomes unavailable or is deleted.

Relevant config files

The **odf-operator** comes with a **ConfigMap** of variables that can be used to modify the behavior of the operator.

Relevant log files

To get an understanding of the OpenShift Data Foundation and troubleshoot issues, you can look at the following:

- Operator Pod logs
- StorageCluster status
- Operator Pod logs
  Each operator provides standard Pod logs that include information about reconciliation and errors encountered. These logs often have information about successful reconciliation which can be filtered out and ignored.
- StorageCluster status and events
  The storageCluster CR stores the reconciliation details in the status of the CR. The spec of the storage cluster contains the name, namespace, and Kind of the actual storage cluster CRD, which the administrator can use to find further information on the status of the storage cluster.

Lifecycle

The **odf-operator** is required to be present as long as the OpenShift Data Foundation bundle remains installed. This is managed as part of OLM's reconciliation of the OpenShift Data Foundation CSV. At least one instance of the pod should be in **Ready** state.

The operator operands such as CRDs should not affect the lifecycle of the operator. The creation and deletion of **StorageClusters** is an operation outside the operator's control and must be initiated by the administrator or automated with the appropriate application programming interface (API) calls.

3.2. OPENSHIFT CONTAINER STORAGE OPERATOR

The **ocs-operator** can be described as a "meta" operator for OpenShift Data Foundation, that is, an operator meant to influence other operators and serves as a configuration gateway for the features provided by the other operators. It does not directly manage the other operators.

The **ocs-operator** has the following primary functions:

- Creates Custom Resources (CRs) that trigger the other operators to reconcile against them.
- Abstracts the Ceph and Multicloud Object Gateway configurations and limits them to known best practices that are validated and supported by Red Hat.
- Creates and reconciles the resources required to deploy containerized Ceph and NooBaa according to the support policies.

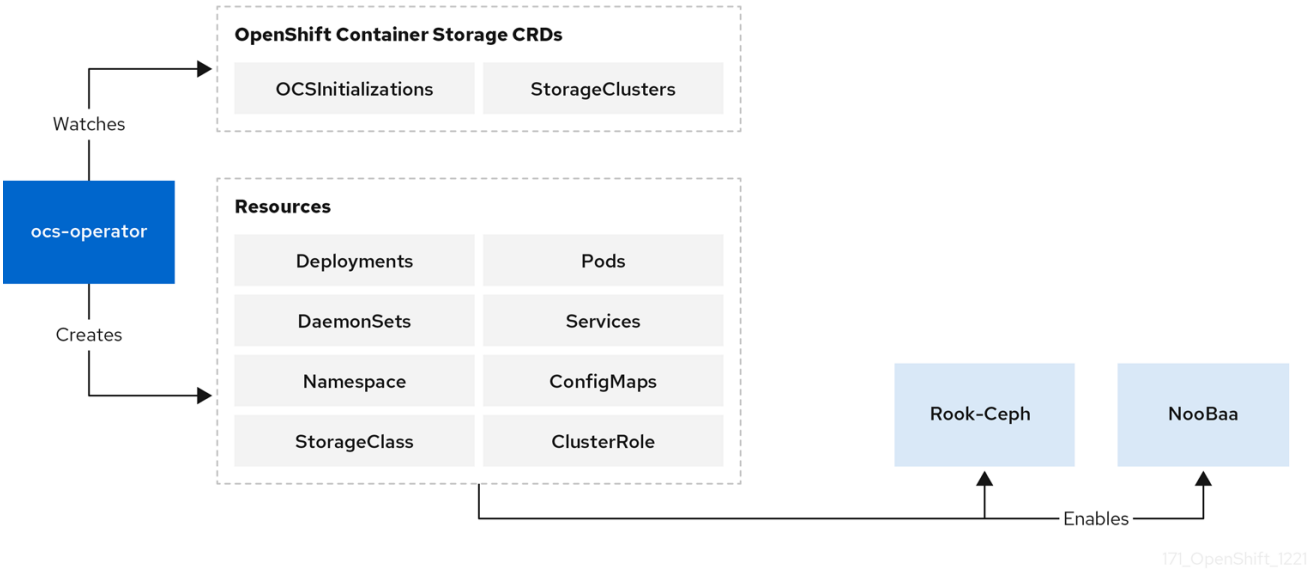
3.2.1. Components

The **ocs-operator** does not have any dependent components. However, the operator has a dependency on the existence of all the custom resource definitions (CRDs) from other operators, which are defined in the **ClusterServiceVersion** (CSV).

3.2.2. Design diagram

The OpenShift Container Storage operator design diagram shows how OpenShift Container Storage is integrated with the OpenShift Container Platform.

Figure 3.1. OpenShift Container Storage Operator



3.2.3. Responsibilities

The **ocs-operator** is responsible for two CRDs.

- **OCSInitialization**
- **StorageCluster**

OCSInitialization is a singleton CRD used for encapsulating operations that apply at the operator level. The operator takes care of ensuring that one instance always exists. The CR triggers the following:

- Performs initialization tasks required for OpenShift Container Storage. If needed, these tasks can be triggered to run again by deleting the **OCSInitialization** CRD.
    - Ensures that the required Security Context Constraints (SCCs) for OpenShift Container Storage are present.
- Manages the deployment of the Ceph toolbox Pod, used for performing advanced troubleshooting and recovery operations.

The **StorageCluster** CRD represents the system that provides the full functionality of OpenShift Container Storage. It triggers the operator to ensure the generation and reconciliation of **Rook-Ceph** and **NooBaa** CRDs. The **ocs-operator** algorithmically generates the **CephCluster** and **NooBaa** CRDs based on the configuration in the **StorageCluster** spec. The operator also creates additional CRs, such as **CephBlockPools**, **Routes**, and so on. These resources are required for enabling different features of OpenShift Container Storage. Currently, only one StorageCluster CR per OpenShift Container Platform cluster is supported.

3.2.4. Resources

The **ocs-operator** creates the following CRs in response to the spec of the CRDs it defines. The configuration of some of these resources can be overridden, allowing for changes to the generated spec or not creating them altogether.

General resources

Events

Creates various events when required in response to reconciliation.

Persistent Volumes (PVs)

PVs are not created directly by the operator. However, the operator keeps track of all the PVs created by the Ceph CSI drivers and ensures that the PVs have appropriate annotations for the supported features.

Quickstarts

Deploys various Quickstart CRs for the OpenShift Container Platform Console.

Rook-Ceph resources

CephBlockPool

Define the default Ceph block pools. **CephFilesysPrometheusRulesoute** for the Ceph object store.

StorageClass

Define the default Storage classes. For example, for **CephBlockPool** and **CephFilesystem**).

VolumeSnapshotClass

Define the default volume snapshot classes for the corresponding storage classes.

Multicloud Object Gateway resources

NooBaa

Define the default Multicloud Object Gateway system.

Monitoring resources

- Metrics Exporter Service

- Metrics Exporter Service Monitor
- PrometheusRules

3.2.5. Limitation

The **ocs-operator** neither deploys nor reconciles the other Pods of OpenShift Data Foundation. The **ocs-operator** CSV defines the top-level components such as operator Deployments and the Operator Lifecycle Manager (OLM) reconciles the specified component.

3.2.6. High availability

High availability is not a primary requirement for the **ocs-operator** pod, similar to most of the other operators. In general, there are no operations that require or benefit from process distribution. OpenShift Container Platform quickly spins up a replacement pod whenever the current operator pod becomes unavailable or is deleted.

3.2.7. Relevant config files

The **ocs-operator** configuration is entirely specified by the CSV and is not modifiable without a custom build of the CSV.

3.2.8. Relevant log files

To get an understanding of the OpenShift Container Storage and troubleshoot issues, you can look at the following:

- Operator Pod logs
- StorageCluster status and events
- OCSInitialization status

Operator Pod logs

Each operator provides standard Pod logs that include information about reconciliation and errors encountered. These logs often have information about successful reconciliation which can be filtered out and ignored.

StorageCluster status and events

The **StorageCluster** CR stores the reconciliation details in the status of the CR and has associated events. Status contains a section of the expected container images. It shows the container images that it expects to be present in the pods from other operators and the images that it currently detects. This helps to determine whether the OpenShift Container Storage upgrade is complete.

OCSInitialization status This status shows whether the initialization tasks are completed successfully.

3.2.9. Lifecycle

The **ocs-operator** is required to be present as long as the OpenShift Container Storage bundle remains installed. This is managed as part of OLM's reconciliation of the OpenShift Container Storage CSV. At least one instance of the pod should be in **Ready** state.

The operator operands such as CRDs should not affect the lifecycle of the operator. An **OCSInitialization** CR should always exist. The operator creates one if it does not exist. The creation and deletion of StorageClusters is an operation outside the operator's control and must be initiated by the administrator or automated with the appropriate API calls.

3.3. ROOK-CEPH OPERATOR

Rook-Ceph operator is the Rook operator for Ceph in the OpenShift Data Foundation. Rook enables Ceph storage systems to run on the OpenShift Container Platform.

The Rook-Ceph operator is a simple container that automatically bootstraps the storage clusters and monitors the storage daemons to ensure the storage clusters are healthy.

3.3.1. Components

The Rook-Ceph operator manages a number of components as part of the OpenShift Data Foundation deployment.

Ceph daemons

Mons

The monitors (mons) provide the core metadata store for Ceph.

OSDs

The object storage daemons (OSDs) store the data on underlying devices.

Mgr

The manager (mgr) collects metrics and provides other internal functions for Ceph.

RGW

The RADOS Gateway (RGW) provides the S3 endpoint to the object store.

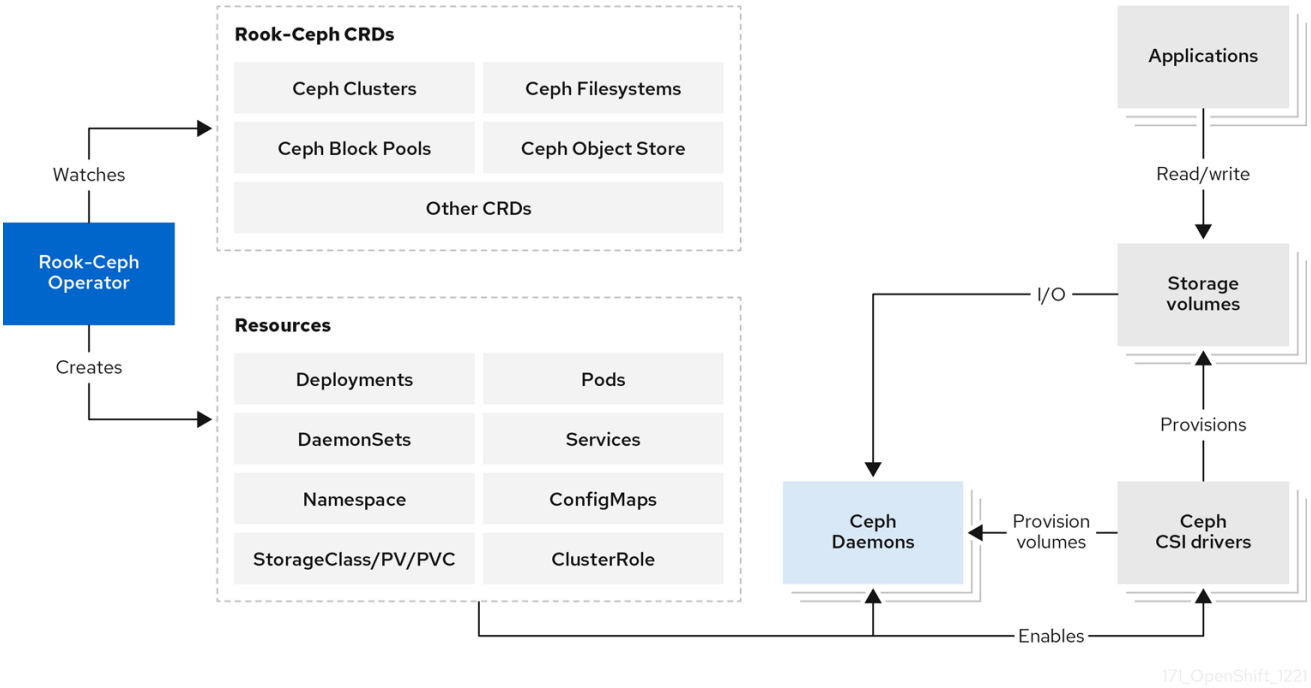
MDS

The metadata server (MDS) provides CephFS shared volumes.

3.3.2. Design diagram

The Rook-Ceph operator design diagram illustrates how Ceph Rook integrates with OpenShift Container Platform.

Figure 3.2. Rook-Ceph Operator



With Ceph running in the OpenShift Container Platform cluster, OpenShift Container Platform applications can mount block devices and filesystems managed by Rook-Ceph, or can use the S3/Swift API for object storage.

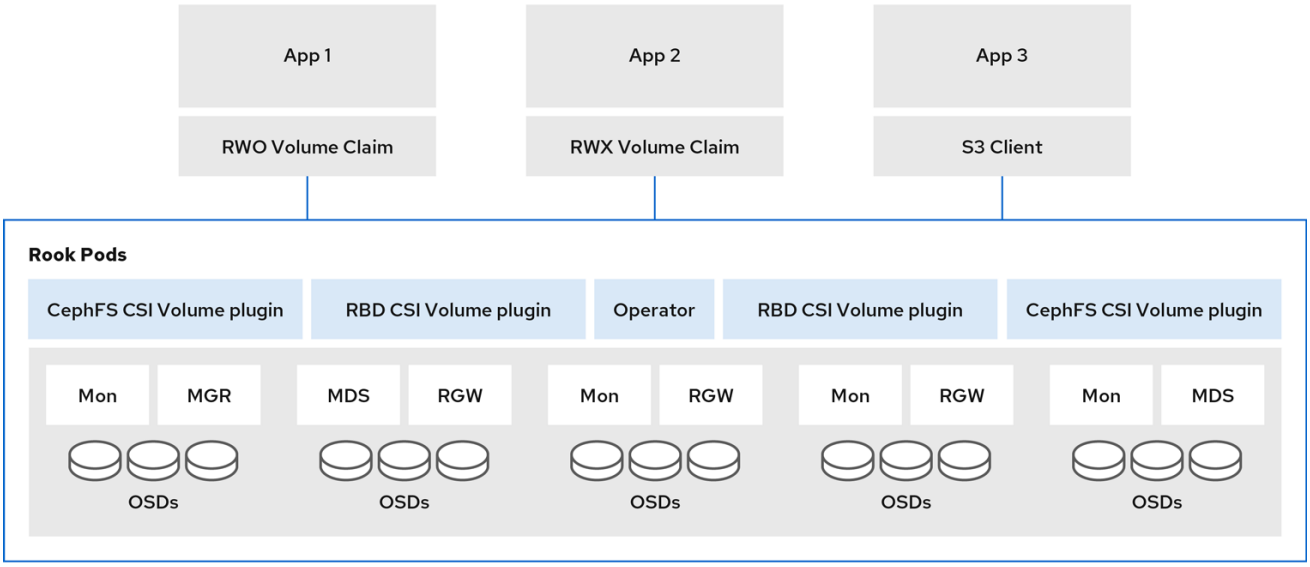
3.3.3. Responsibilities

The Rook-Ceph operator is responsible for configuring and maintaining all Ceph daemons.

The Rook-Ceph operator is a container that bootstraps and monitors the storage cluster. It performs the following functions:

- Automates the configuration of storage components
- Starts, monitors, and manages the Ceph monitor pods and Ceph OSD daemons to provide the RADOS storage cluster
- Initializes the pods and other artifacts to run the services to manage:
  - CRDs for pools
  - Object stores (S3/Swift)
  - Filesystems
- Monitors the Ceph mons and OSDs to ensure that the storage remains available and healthy
- Deploys and manages Ceph mons placement while adjusting the mon configuration based on cluster size
- Watches the desired state changes requested by the API service and applies the changes
- Initializes the Ceph-CSI drivers that are needed for consuming the storage
- Automatically configures the Ceph-CSI driver to mount the storage to pods

Rook-Ceph Operator architecture



171_OpenShift_1221

The Rook-Ceph operator image includes all required tools to manage the cluster. There is no change to the data path. However, the operator does not expose all Ceph configurations. Many of the Ceph features like placement groups and crush maps are hidden from the users and are provided with a better user experience in terms of physical resources, pools, volumes, filesystems, and buckets.

3.3.4. Resources

Rook-Ceph operator adds owner references to all the resources it creates in the **openshift-storage** namespace. When the cluster is uninstalled, the owner references ensure that the resources are all cleaned up. This includes OpenShift Container Platform resources such as **configmaps**, **secrets**, **services**, **deployments**, **daemonsets**, and so on.

The Rook-Ceph operator watches CRs to configure the settings determined by OpenShift Data Foundation, which includes **CephCluster**, **CephObjectStore**, **CephFilesystem**, and **CephBlockPool**.

3.3.5. Lifecycle

Rook-Ceph operator manages the lifecycle of the following pods in the Ceph cluster:

Rook operator

A single pod that owns the reconcile of the cluster.

RBD CSI Driver

- Two provisioner pods, managed by a single deployment.
- One plugin pod per node, managed by a **daemonset**.

CephFS CSI Driver

- Two provisioner pods, managed by a single deployment.
- One plugin pod per node, managed by a **daemonset**.

Monitors (mons)

Three mon pods, each with its own deployment.

Stretch clusters

Contain five mon pods, one in the arbiter zone and two in each of the other two data zones.

Manager (mgr)

There is a single mgr pod for the cluster.

Stretch clusters

There are two mgr pods (starting with OpenShift Data Foundation 4.8), one in each of the two non-arbiter zones.

Object storage daemons (OSDs)

At least three OSDs are created initially in the cluster. More OSDs are added when the cluster is expanded.

Metadata server (MDS)

The CephFS metadata server has a single pod.

RADOS gateway (RGW)

The Ceph RGW daemon has a single pod.

3.4. MCG OPERATOR

The Multicloud Object Gateway (MCG) operator is an operator for OpenShift Data Foundation along with the OpenShift Data Foundation operator and the Rook-Ceph operator. The MCG operator is available upstream as a standalone operator.

The MCG operator performs the following primary functions:

- Controls and reconciles the Multicloud Object Gateway (MCG) component within OpenShift Data Foundation.
- Manages new user resources such as object bucket claims, bucket classes, and backing stores.
- Creates the default out-of-the-box resources.

A few configurations and information are passed to the MCG operator through the OpenShift Data Foundation operator.

3.4.1. Components

The MCG operator does not have sub-components. However, it consists of a reconcile loop for the different resources that are controlled by it.

The MCG operator has a command-line interface (CLI) and is available as a part of OpenShift Data Foundation. It enables the creation, deletion, and querying of various resources. This CLI adds a layer of input sanitation and status validation before the configurations are applied unlike applying a YAML file directly.

3.4.2. Responsibilities and resources

The MCG operator reconciles and is responsible for the custom resource definitions (CRDs) and OpenShift Container Platform entities.

- Backing store
- Namespace store
- Bucket class
- Object bucket claims (OBCs)
- NooBaa, pod stateful sets CRD
- Prometheus Rules and Service Monitoring
- Horizontal pod autoscaler (HPA)

Backing store

A resource that the customer has connected to the MCG component. This resource provides MCG the ability to save the data of the provisioned buckets on top of it.

A default backing store is created as part of the deployment depending on the platform that the OpenShift Container Platform is running on. For example, when OpenShift Container Platform or OpenShift Data Foundation is deployed on Amazon Web Services (AWS), it results in a default backing store which is an AWS::S3 bucket. Similarly, for Microsoft Azure, the default backing store is a blob container and so on.

The default backing stores are created using CRDs for the cloud credential operator, which comes with OpenShift Container Platform. There is no limit on the amount of the backing stores that can be added to MCG. The backing stores are used in the bucket class CRD to define the different policies of the bucket. Refer the documentation of the specific OpenShift Data Foundation version to identify the types of services or resources supported as backing stores.

Namespace store Resources that are used in namespace buckets. No default is created during deployment.

Bucketclass A default or initial policy for a newly provisioned bucket. The following policies are set in a bucketclass:

Placement policy

Indicates the backing stores to be attached to the bucket and used to write the data of the bucket. This policy is used for data buckets and for cache policies to indicate the local cache placement. There are two modes of placement policy:

- Spread. Strips the data across the defined backing stores
- Mirror. Creates a full replica on each backing store

Namespace policy

A policy for the namespace buckets that defines the resources that are being used for aggregation and the resource used for the write target.

Cache Policy

This is a policy for the bucket and sets the hub (the source of truth) and the time to live (TTL) for the cache items.

A default bucket class is created during deployment and it is set with a placement policy that uses the default backing store. There is no limit to the number of bucket class that can be added.

Refer to the documentation of the specific OpenShift Data Foundation version to identify the types of policies that are supported.

Object bucket claims (OBCs)

CRDs that enable provisioning of S3 buckets. With MCG, OBCs receive an optional bucket class to note the initial configuration of the bucket. If a bucket class is not provided, the default bucket class is used.

NooBaa, pod stateful sets CRD

An internal CRD that controls the different pods of the NooBaa deployment such as the DB pod, the core pod, and the endpoints. This CRD must not be changed as it is internal. This operator reconciles the following entities:

- DB pod SCC
- Role Binding and Service Account to allow SSO single sign-on between OpenShift Container Platform and NooBaa user interfaces
- Route for S3 access
- Certificates that are taken and signed by the OpenShift Container Platform and are set on the S3 route

DB pods

The MCG DB pods serve as the storage layer for MCG metadata. They are configured to enable high availability (HA) with a primary-secondary architecture. The primary pod handles write operations and most reads, while the secondary pod maintains a synchronized replica for redundancy and automatic failover. The secondary pod also handles certain read-only operations, reducing the load from the primary pod.

Prometheus rules and service monitoring These CRDs set up scraping points for Prometheus and alert rules that are supported by MCG.

Horizontal pod autoscaler (HPA) It is Integrated with the MCG endpoints. The endpoint pods scale up and down according to CPU pressure (amount of S3 traffic).

3.4.3. High availability

As an operator, the only high availability provided is that the OpenShift Container Platform reschedules a failed pod.

3.4.4. Relevant log files

To troubleshoot issues with the NooBaa operator, you can look at the following:

- Operator pod logs, which are also available through the must-gather.
- Different CRDs or entities and their statuses that are available through the must-gather.

3.4.5. Lifecycle

The MCG operator runs and reconciles after OpenShift Data Foundation is deployed and until it is uninstalled.

CHAPTER 4. OPENSHIFT DATA FOUNDATION INSTALLATION OVERVIEW

OpenShift Data Foundation consists of multiple components managed by multiple operators.



NOTE

Red Hat OpenShift Data Foundation supports Technical Supportability Review with AI, which analyzes cluster diagnostic data to identify configuration risks, supportability concerns, and remediation opportunities. Consider running a review after deployment and also before major events such as upgrades, cluster expansions. For more information, see the knowledgebase article Red Hat Technical Supportability Review with AI .

4.1. INSTALLED OPERATORS

When you install OpenShift Data Foundation from the Software Catalog, the following four separate Deployments are created:

- **odf-operator**: Defines the **odf-operator** Pod
- **ocs-operator**: Defines the **ocs-operator** Pod which runs processes for **ocs-operator** and its **metrics-exporter**.
- **rook-ceph-operator**: Defines the **rook-ceph-operator** Pod.
- **mcg-operator**: Defines the **mcg-operator** Pod.

These operators run independently and interact with each other by creating customer resources (CRs) watched by the other operators. The **ocs-operator** is primarily responsible for creating the CRs to configure Ceph storage and Multicloud Object Gateway. The **mcg-operator** sometimes creates Ceph volumes for use by its components.

4.2. OPENSHIFT CONTAINER STORAGE INITIALIZATION

The OpenShift Data Foundation bundle also defines an external plugin to the OpenShift Container Platform Console, adding new screens and functionality not otherwise available in the Console. This plugin runs as a web server in the **odf-console-plugin** Pod, which is managed by a Deployment created by the OLM at the time of installation.

The **ocs-operator** automatically creates an **OCSInitialization** CR after it gets created. Only one **OCSInitialization** CR exists at any point in time. It controls the **ocs-operator** behaviors that are not restricted to the scope of a single **StorageCluster**, but only performs them once. When you delete the **OCSInitialization** CR, the **ocs-operator** creates it again and this allows you to re-trigger its initialization operations.

The **OCSInitialization** CR controls the following behaviors:

SecurityContextConstraints (SCCs)

After the **OCSInitialization** CR is created, the **ocs-operator** creates various SCCs for use by the component Pods.

Ceph Toolbox Deployment

You can use the **OCSInitialization** to deploy the Ceph Toolbox Pod for the advanced Ceph operations.

Rook-Ceph Operator Configuration

This configuration creates the **rook-ceph-operator-config ConfigMap** that governs the overall configuration for **rook-ceph-operator** behavior.

4.3. STORAGE CLUSTER CREATION

The OpenShift Data Foundation operators themselves provide no storage functionality, and the desired storage configuration must be defined.

After you install the operators, create a new **StorageCluster**, using either the OpenShift Container Platform console wizard or the CLI and the **ocs-operator** reconciles this **StorageCluster**. OpenShift Data Foundation supports a single **StorageCluster** per installation. Any **StorageCluster** CRs created after the first one are ignored by **ocs-operator** reconciliation.

OpenShift Data Foundation allows the following StorageCluster configurations:

Internal

In the Internal mode, all the components run containerized within the OpenShift Container Platform cluster and use dynamically provisioned persistent volumes (PVs) created against the **StorageClass** specified by the administrator in the installation wizard.

Internal-attached

This mode is similar to the Internal mode but the administrator is required to define the local storage devices directly attached to the cluster nodes that the Ceph uses for its backing storage. Also, the administrator needs to create the CRs that the local storage operator reconciles to provide the **StorageClass**. The **ocs-operator** uses this **StorageClass** as the backing storage for Ceph.

External

In this mode, Ceph components do not run inside the OpenShift Container Platform cluster; instead connectivity is provided to an external OpenShift Container Storage installation for which the applications can create PVs. The other components run within the cluster as required.

MCG Standalone

This mode facilitates the installation of a Multicloud Object Gateway system without an accompanying CephCluster.

After a **StorageCluster** CR is found, **ocs-operator** validates it and begins to create subsequent resources to define the storage components.

4.3.1. Internal mode storage cluster

In internal and internal-attached modes, the **ocs-operator** creates storage classes, snapshot classes, block and file system resources, and the CephCluster and NooBaa CRs that drive the full Ceph and Multicloud Object Gateway deployment on your OpenShift Data Foundation cluster.

Both internal and internal-attached storage clusters have the same setup process as follows:

StorageClasses	Create the storage classes that cluster applications use to create Ceph volumes.

SnapshotClasses	Create the volume snapshot classes that the cluster applications use to create snapshots of Ceph volumes.
Ceph RGW configuration	Create various Ceph object CRs to enable and provide access to the Ceph RGW object storage endpoint.
Ceph RBD Configuration	Create the **CephBlockPool** CR to enable RBD storage.
CephFS Configuration	Create the **CephFilesystem** CR to enable CephFS storage.
Rook-Ceph Configuration	Create the **rook-config-override ConfigMap** that governs the overall behavior of the underlying Ceph cluster.
CephCluster	Create the **CephCluster** CR to trigger Ceph reconciliation from **rook-ceph-operator**. For more information, see Rook-Ceph operator.
NoobaaSystem	Create the **NooBaa** CR to trigger reconciliation from **mcg-operator**. For more information, see MCG operator.
Job templates	Create **OpenShift Template** CRs that define Jobs to run administrative operations for OpenShift Container Storage.
Quickstarts	Create the **QuickStart** CRs that display the quickstart guides in the Web Console.



NOTE

The RGW (RADOS Gateway) component is deployed only in on-premises environments. It is not created for cloud-based deployments.

4.3.1.1. Cluster Creation

After the **ocs-operator** creates the **CephCluster** CR, the **rook-operator** creates the Ceph cluster according to the desired configuration. The **rook-operator** configures the following components:

Ceph **mon** daemons	Three Ceph **mon** daemons are started on different nodes in the cluster. They manage the core metadata for the Ceph cluster and they must form a majority quorum. The metadata for each **mon** is backed either by a PV if it is in a cloud environment or a path on the local host if it is in a local storage device environment.
Ceph **mgr** daemon	This daemon is started and it gathers metrics for the cluster and report them to Prometheus.

Ceph OSDs	These OSDs are created according to the configuration of the **storageClassDeviceSets**. Each OSD consumes a PV that stores the user data. By default, Ceph maintains three replicas of the application data across different OSDs for high durability and availability using the **CRUSH** algorithm.
CSI provisioners	These provisioners are started for RBD and **CephFS**. When volumes are requested for the storage classes of OpenShift Container Storage, the requests are directed to the **Ceph-CSI** driver to provision the volumes in Ceph.
CSI volume plugins and **CephFS**	The CSI volume plugins for RBD and **CephFS** are started on each node in the cluster. The volume plugin needs to be running wherever the Ceph volumes are required to be mounted by the applications.

After the **CephCluster** CR is configured, Rook reconciles the remaining Ceph CRs to complete the setup:

CephBlockPool	The **CephBlockPool** CR provides the configuration for Rook operator to create Ceph pools for RWO volumes.
CephFilesystem	The **CephFilesystem** CR instructs the Rook operator to configure a shared file system with CephFS, typically for RWX volumes. The CephFS metadata server (MDS) is started to manage the shared volumes.
CephObjectStore	The **CephObjectStore** CR instructs the Rook operator to configure an object store with the RGW service. It can also be used to provide RGW configuration settings for internal object stores, including secret-backed settings by using **rgwConfigFromSecret**.
CephObjectStoreUser CR	The **CephObjectStoreUser** CR instructs the Rook operator to configure an object store user for NooBaa to consume, publishing **access/private** key as well as the **CephObjectStore** endpoint.

The operator monitors the Ceph health to ensure that storage platform remains healthy. If a **mon** daemon goes down for too long a period (10 minutes), Rook starts a new **mon** in its place so that the full quorum can be fully restored.

When the **ocs-operator** updates the **CephCluster** CR, Rook immediately responds to the requested changes to update the cluster configuration.

4.3.1.2. NooBaa System creation

When a NooBaa system is created, the **mcg-operator** automatically reconciles a platform-appropriate default backing store, a default bucket class, the core database and endpoint pods, and the S3 route and service that applications use to access object storage.

Default BackingStore Depending on the platform that OpenShift Container Platform and OpenShift Data Foundation are deployed on, a default backing store resource is created so that buckets can use it for their placement policy. The different options are as follows:

Amazon Web Services (AWS) deployment	The **mcg-operator** uses the **CloudCredentialsOperator** (CCO) to mint credentials in order to create a new AWS::S3 bucket and creates a **BackingStore** on top of that bucket.
Microsoft Azure deployment	The **mcg-operator** uses the CCO to mint credentials in order to create a new Azure Blob and creates a **BackingStore** on top of that bucket.
Google Cloud Platform (GCP) deployment	The **mcg-operator** uses the CCO to mint credentials in order to create a new GCP bucket and will create a BackingStore on top of that bucket.
On-prem deployment	If RGW exists, the **mcg-operator** creates a new **CephUser** and a new bucket on top of RGW and creates a **BackingStore** on top of that bucket.
None of the previously mentioned deployments are applicable	The **mcg-operator** creates a **pv-pool** based on the default storage class and creates a **BackingStore** on top of that bucket.

Default BucketClass

A **BucketClass** with a placement policy to the default **BackingStore** is created.

NooBaa pods The following NooBaa pods are created and started:

Database (DB)	This is a Postgres DB holding metadata, statistics, events, and so on. However, it does not hold the actual data being stored.
Core	This is the pod that handles configuration, background processes, metadata management, statistics, and so on.
Endpoints	These pods perform the actual I/O-related work such as deduplication and compression, communicating with different services to write and read data, and so on. The endpoints are integrated with the **HorizonalPodAutoscaler** and their number increases and decreases according to the CPU usage observed on the existing endpoint pods.

Route A Route for the NooBaa S3 interface is created for applications that use S3.

Service A Service for the NooBaa S3 interface is created for applications that use S3.

4.3.2. External mode storage cluster

For external storage clusters, **ocs-operator** follows a slightly different setup process. The **ocs-operator** looks for the existence of the **rook-ceph-external-cluster-details ConfigMap**, which must be created by someone else, either the administrator or the Console.

For information about how to create the **ConfigMap**, see Creating an OpenShift Data Foundation Cluster for external mode. The **ocs-operator** then creates some or all of the following resources, as specified in the **ConfigMap**:

External Ceph Configuration	A ConfigMap that specifies the endpoints of the external **mons**.
External Ceph Credentials Secret	A Secret that contains the credentials to connect to the external Ceph instance.
External Ceph StorageClasses	One or more StorageClasses to enable the creation of volumes for RBD, CephFS, and/or RGW.
Enable CephFS CSI Driver	If a **CephFS** StorageClass is specified, configure **rook-ceph-operator** to deploy the **CephFS** CSI Pods.
Ceph RGW Configuration	If an RGW StorageClass is specified, create various Ceph Object CRs to enable and provide access to the Ceph RGW object storage endpoint.

After creating the resources specified in the **ConfigMap**, the StorageCluster creation process proceeds as follows:

CephCluster	Create the **CephCluster** CR to trigger Ceph reconciliation from**rook-ceph-operator** (see subsequent sections).
SnapshotClasses	Create the **SnapshotClasses** that applications use to create snapshots of Ceph volumes.
NoobaaSystem	Create the **NooBaa** CR to trigger reconciliation from the noobaa-operator (see subsequent sections).
QuickStarts	Create the **Quickstart** CRs that display the quickstart guides in the Console.

4.3.2.1. Cluster Creation

The Rook operator performs the following operations when the **CephCluster** CR is created in external mode:

- The operator validates that a connection is available to the remote Ceph cluster. The connection requires **mon** endpoints and secrets to be imported into the local cluster.
- The CSI driver is configured with the remote connection to Ceph. The RBD and **CephFS** provisioners and volume plugins are started similarly to the CSI driver when configured in internal mode, the connection to Ceph happens to be external to the OpenShift cluster.
- Periodically watch for monitor address changes and update the Ceph-CSI configuration accordingly.

4.3.2.2. NooBaa System creation

When a NooBaa system is created, the **mcg-operator** automatically reconciles a platform-appropriate default backing store, a default bucket class, the core database and endpoint pods, and the S3 route and service that applications use to access object storage.

Default BackingStore Depending on the platform that OpenShift Container Platform and OpenShift Data Foundation are deployed on, a default backing store resource is created so that buckets can use it for their placement policy. The different options are as follows:

Amazon Web Services (AWS) deployment	The **mcg-operator** uses the **CloudCredentialsOperator** (CCO) to mint credentials in order to create a new AWS::S3 bucket and creates a **BackingStore** on top of that bucket.
Microsoft Azure deployment	The **mcg-operator** uses the CCO to mint credentials in order to create a new Azure Blob and creates a **BackingStore** on top of that bucket.
Google Cloud Platform (GCP) deployment	The **mcg-operator** uses the CCO to mint credentials in order to create a new GCP bucket and will create a BackingStore on top of that bucket.
On-prem deployment	If RGW exists, the **mcg-operator** creates a new **CephUser** and a new bucket on top of RGW and creates a **BackingStore** on top of that bucket.
None of the previously mentioned deployments are applicable	The **mcg-operator** creates a **pv-pool** based on the default storage class and creates a **BackingStore** on top of that bucket.

Default BucketClass

A **BucketClass** with a placement policy to the default **BackingStore** is created.

NooBaa pods The following NooBaa pods are created and started:

Database (DB)	This is a Postgres DB holding metadata, statistics, events, and so on. However, it does not hold the actual data being stored.
Core	This is the pod that handles configuration, background processes, metadata management, statistics, and so on.
Endpoints	These pods perform the actual I/O-related work such as deduplication and compression, communicating with different services to write and read data, and so on. The endpoints are integrated with the **HorizonalPodAutoscaler** and their number increases and decreases according to the CPU usage observed on the existing endpoint pods.

Route A Route for the NooBaa S3 interface is created for applications that use S3.

Service A Service for the NooBaa S3 interface is created for applications that use S3.

4.3.3. MCG Standalone StorageCluster

In this mode, no CephCluster is created. Instead a NooBaa system CR is created using default values to take advantage of pre-existing StorageClasses in the OpenShift Container Platform. dashboards.

4.3.3.1. NooBaa System creation

When a NooBaa system is created, the **mcg-operator** automatically reconciles a platform-appropriate default backing store, a default bucket class, the core database and endpoint pods, and the S3 route and service that applications use to access object storage.

Default BackingStore Depending on the platform that OpenShift Container Platform and OpenShift Data Foundation are deployed on, a default backing store resource is created so that buckets can use it for their placement policy. The different options are as follows:

Amazon Web Services (AWS) deployment	The **mcg-operator** uses the **CloudCredentialsOperator** (CCO) to mint credentials in order to create a new AWS::S3 bucket and creates a **BackingStore** on top of that bucket.
Microsoft Azure deployment	The **mcg-operator** uses the CCO to mint credentials in order to create a new Azure Blob and creates a **BackingStore** on top of that bucket.
Google Cloud Platform (GCP) deployment	The **mcg-operator** uses the CCO to mint credentials in order to create a new GCP bucket and will create a BackingStore on top of that bucket.
On-prem deployment	If RGW exists, the **mcg-operator** creates a new**CephUser** and a new bucket on top of RGW and creates a **BackingStore** on top of that bucket.

None of the previously mentioned deployments are applicable	The **mcg-operator** creates a **pv-pool** based on the default storage class and creates a **BackingStore** on top of that bucket.

Default BucketClass

A **BucketClass** with a placement policy to the default **BackingStore** is created.

NooBaa pods The following NooBaa pods are created and started:

Database (DB)	This is a Postgres DB holding metadata, statistics, events, and so on. However, it does not hold the actual data being stored.
Core	This is the pod that handles configuration, background processes, metadata management, statistics, and so on.
Endpoints	These pods perform the actual I/O-related work such as deduplication and compression, communicating with different services to write and read data, and so on. The endpoints are integrated with the **HorizonalPodAutoscaler** and their number increases and decreases according to the CPU usage observed on the existing endpoint pods.

Route A Route for the NooBaa S3 interface is created for applications that use S3.

Service A Service for the NooBaa S3 interface is created for applications that use S3.

CHAPTER 5. OPENSHIFT DATA FOUNDATION UPGRADE OVERVIEW

As an operator bundle managed by the Operator Lifecycle Manager (OLM), OpenShift Data Foundation leverages its operators to perform high-level tasks of installing and upgrading the product through **ClusterServiceVersion** (CSV) CRs.

5.1. UPGRADE WORKFLOWS

OpenShift Data Foundation recognizes two types of upgrades: Z-stream release upgrades and Minor Version release upgrades. While the user interface workflows for these two upgrade paths are not quite the same, the resulting behaviors are fairly similar.

The distinctions are as follows:

For Z-stream releases, OCS will publish a new bundle in the **redhat-operators CatalogSource**. The OLM will detect this and create an **InstallPlan** for the new CSV to replace the existing CSV. The Subscription approval strategy, whether Automatic or Manual, will determine whether the OLM proceeds with reconciliation or waits for administrator approval.

For Minor Version releases, OpenShift Container Storage will also publish a new bundle in the **redhat-operators CatalogSource**. The difference is that this bundle will be part of a new channel, and channel upgrades are not automatic. The administrator must explicitly select the new release channel. Once this is done, the OLM will detect this and create an **InstallPlan** for the new CSV to replace the existing CSV. Since the channel switch is a manual operation, OLM will automatically start the reconciliation.

From this point onwards, the upgrade processes are identical.

5.2. CLUSTERSERVICEVERSION RECONCILIATION

When the OLM detects an approved **InstallPlan**, it begins the process of reconciling the CSVs. Broadly, it does this by updating the operator resources based on the new spec, verifying the new CSV installs correctly, then deleting the old CSV. The upgrade process will push updates to the operator Deployments, which will trigger the restart of the operator Pods using the images specified in the new CSV.



NOTE

While it is possible to make changes to a given CSV and have those changes propagate to the relevant resource, when upgrading to a new CSV all custom changes will be lost, as the new CSV will be created based on its unaltered spec.

5.3. OPERATOR RECONCILIATION

At this point, the reconciliation of the OpenShift Data Foundation operands proceeds as defined in the OpenShift Data Foundation installation overview . The operators will ensure that all relevant resources exist in their expected configurations as specified in the user-facing resources (for example, **StorageCluster**).

CHAPTER 6. NETWORKING PORTS

OpenShift Data Foundation requires specific network ports to be accessible depending on the deployment mode. This section describes the network port requirements for both external and internal mode deployments.

6.1. NETWORK PORTS FOR EXTERNAL MODE

When deploying OpenShift Data Foundation in external mode, specific network ports must be open between OpenShift Container Platform and the external Ceph storage cluster to ensure proper communication and functionality.

6.1.1. Network ports required between OpenShift Container Platform and Ceph

The following TCP ports must be accessible from OpenShift Container Platform nodes to the external Red Hat Ceph Storage cluster:

TCP ports	Purpose
6789, 3300	Ceph Monitor
6800 - 7300	Ceph OSD, MGR, MDS
9283	Ceph MGR Prometheus Exporter

For more information about why these ports are required, see *Chapter 2. Ceph network configuration* of the Red Hat Ceph Storage Configuration Guide .

For detailed information about deploying OpenShift Data Foundation in external mode, see Overview of deploying in external mode.

6.2. NETWORK PORTS FOR INTERNAL MODE

When deploying OpenShift Data Foundation in internal mode, the network port requirements depend on the deployment architecture.

6.2.1. Network ports required for internal mode deployment

OpenShift Data Foundation uses pod networking for all communication between components. This means no specific open ports are required between OpenShift Container Platform nodes and the internal Ceph storage cluster.



NOTE

The use of pod networking eliminates the need for dedicated port configurations, simplifying network requirements and improving security.

For more information about Ceph network configuration, see *Chapter 2. Ceph network configuration* of the Red Hat Ceph Storage Configuration Guide .

CHAPTER 7. MULTUS ARCHITECTURE FOR OPENSHIFT DATA FOUNDATION

The OpenShift multi-network plugin (Multus) reference architecture defines the supported configurations for deploying OpenShift Data Foundation with Multus. This architecture provides information about OpenShift Data Foundation requirements, processes, and features that are relevant to Multus.

For configurations that deviate from this reference architecture, open a support exception using the Jira ticket link: https://issues.redhat.com/projects/SUPPORTEX.

7.1. ASSUMPTIONS

This Multus reference architecture section assumes the following:

- OpenShift Data Foundation is deployed in internal mode using dedicated storage nodes.
- OpenShift control plane nodes do not run applications that require OpenShift Data Foundation storage.
  This is not required for supportability, but some node selectors and/or tolerations might need to be modified as needed for other cluster layouts.
- VLANs are present. VLANs are optional and neither recommended nor discouraged. However, in many instances VLANs are required depending on the user environment.

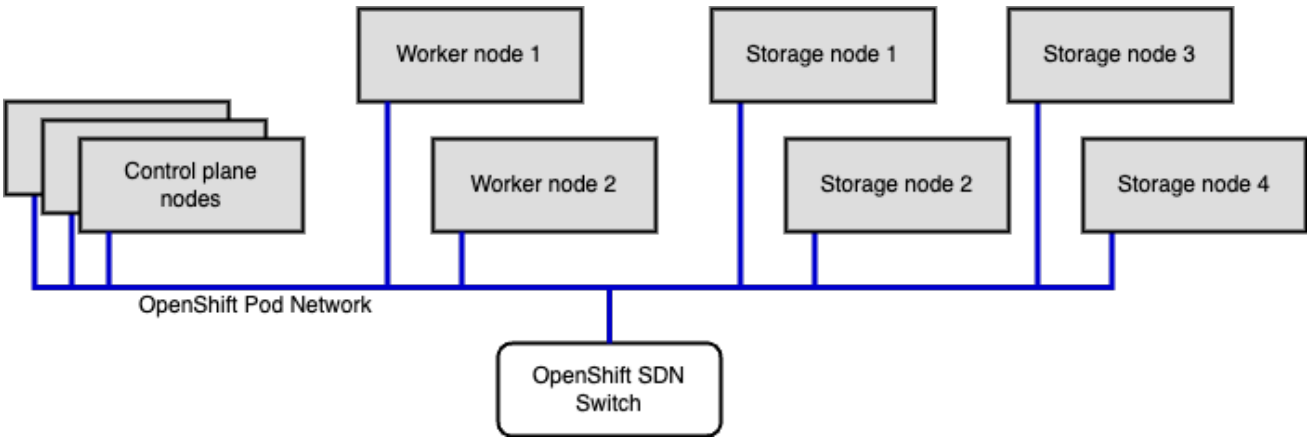
7.2. EXAMPLE SCENARIO USED

This Multus reference architecture uses the following example:

- Four dedicated OpenShift Data Foundation storage nodes
- Two OpenShift worker nodes
- Three Openshift control plane nodes

The following diagram shows the example cluster network architecture in a traditional OpenShift Data Foundation deployment without Multus:

Figure 7.1. Example cluster network architecture without Multus



7.3. HOW TO CHOOSE MULTUS ARCHITECTURE FOR AN OPENSHIFT DATA FOUNDATION DEPLOYMENT

Choosing the right Multus network architecture depends on your performance, bandwidth, and isolation goals. Selecting the correct option before deployment avoids reconfiguration later.

Multus networking for an OpenShift Data Foundation deployment is chosen for one or more of the following reasons:

- To reduce latency of the storage system by avoiding the OpenShift Pod (software) network
- To dedicate bandwidth to the storage system to avoid the noisy neighbor problem between storage and applications
- To segregate the storage network from applications for isolation

Multus is enabled for either or both of the following types of OpenShift Data Foundation storage networks:

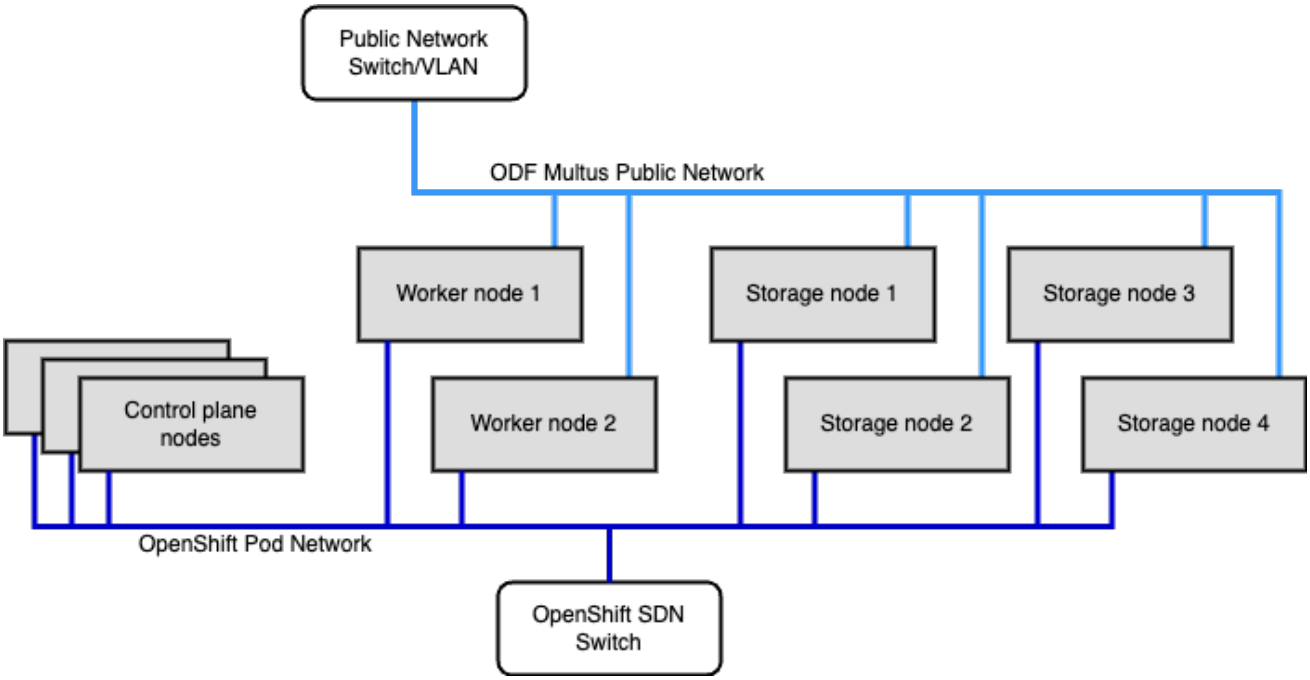
- Public network
- Cluster network

However, it is recommended to configure only the Multus public network as a starting point.

7.3.1. Multus public network

The following diagram shows the cluster network architecture for an OpenShift Data Foundation deployment using the Multus public network:

Figure 7.2. OpenShift Data Foundation deployment using Multus public network



This architecture achieves all the three benefits of Multus networking:

- Storage latency is reduced by avoiding the software-defined pod network
- Storage system gets dedicated bandwidth that avoids the noisy-neighbor interference

- Storage network is isolated from applications that prevent pods from snooping the storage network

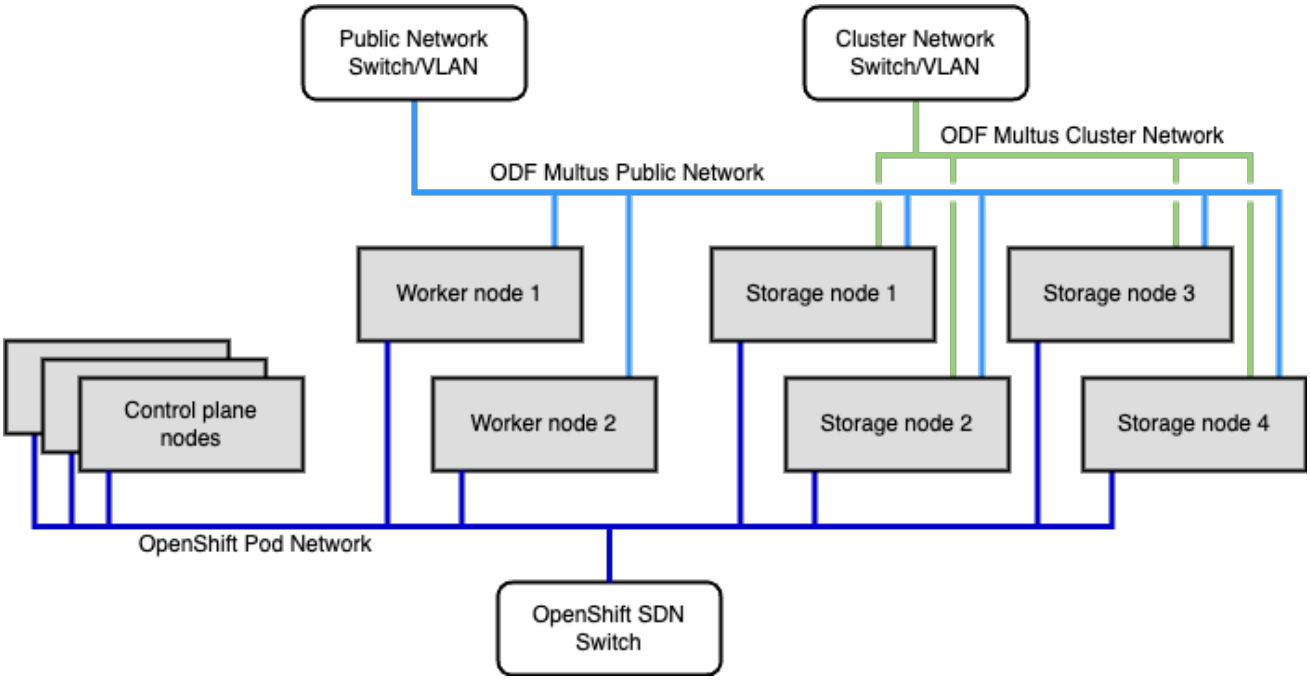
However, if the public network has less throughput than the Pod network, storage traffic might not be faster than a traditional deployment.

Public network is the recommended architecture for the majority of OpenShift Data Foundation cluster deployments. In this architecture, all storage traffic uses the dedicated public network, which includes I/O traffic from applications, data replication traffic within the storage system, and data recovery traffic also within the storage system.

7.3.2. Multus public and cluster networks

The following diagram shows the cluster network architecture for an OpenShift Data Foundation deployment using both Multus public network and Multus cluster network:

Figure 7.3. Cluster network architecture using both Multus public network and Multus cluster network



When both public and cluster networks are used simultaneously, all three benefits of Multus are achieved. In this case, each storage network performs a specific role in the OpenShift Data Foundation cluster. The public network handles only the I/O traffic from applications and the cluster network handles storage replication and recovery traffic.

The cluster network handles data replication traffic on a regular basis. Assuming OpenShift Data Foundation's default 3x replication, the cluster network handles 2x the application I/O traffic for each data write to bring the total number of storage data copies up to 3x. During a storage node or zone failure event, the cluster network also handles recovering data in the cluster.

In this architecture, the size of the cluster network is recommended to be 2-3x larger than the public network. Larger sizing keeps the failover times short, but it also results in a large amount of bandwidth being wasted during normal cluster operations. Overall storage bandwidth during a storage failure event might be limited by either the public or cluster network depending on the size of either network.

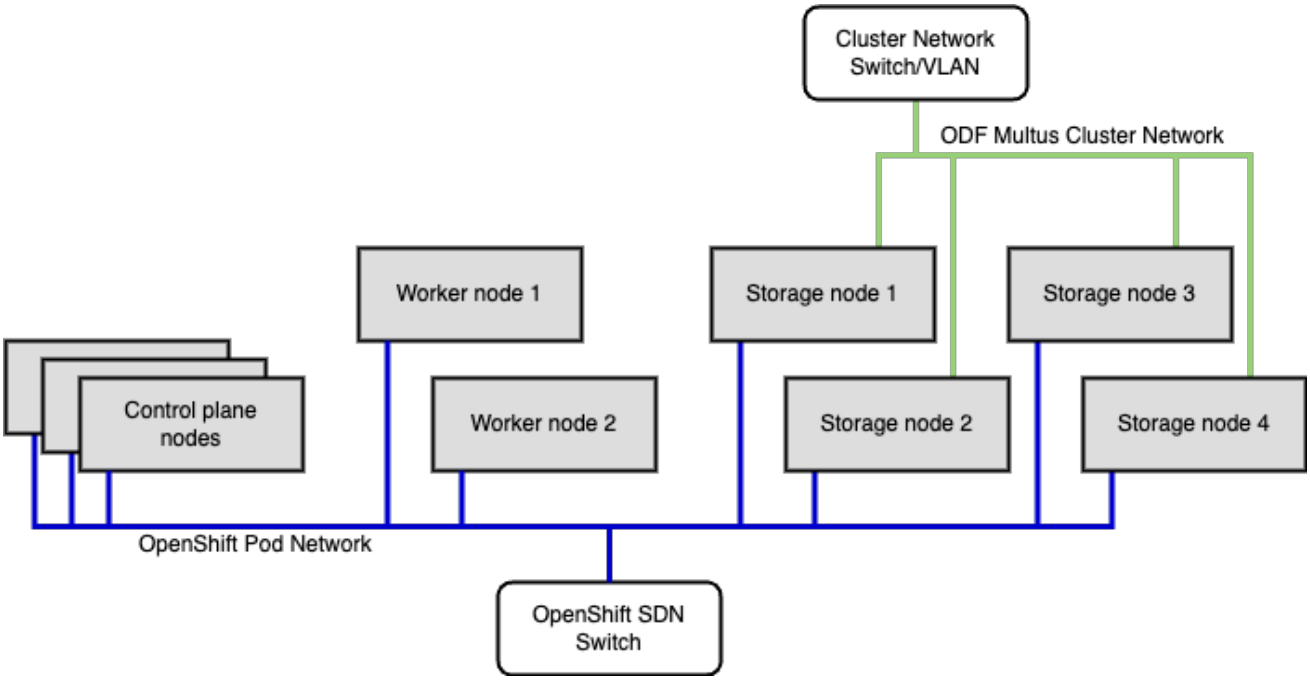
This architecture is more complicated than the recommended public-only architecture and has many more factors that could affect user SLOs. Hence, as an alternative, OpenShift Data Foundation suggests bonding all storage network interfaces together to create a single, larger public network

instead.

7.3.3. Multus cluster network

The following diagram shows the cluster network architecture for an OpenShift Data Foundation deployment using Multus cluster network:

Figure 7.4. Cluster network architecture



In the cluster network architecture, Multus benefits are partially achieved. The cluster network takes the replication and recovery roles. The OpenShift Pod network takes the role of handling storage I/O from applications. This ensures noisy-neighbor interference is minimized, but application I/O can still be affected by other application networking, and vice versa.

Application I/O to the storage system is expected to have higher latency than other Multus architectures due to the software-defined Pod network. Similarly, the storage network is not isolated from applications. For example, a privileged Pod could still snoop storage traffic.

7.4. MULTUS NETWORKING REQUIREMENTS

Before configuring Multus networking for OpenShift Data Foundation, your environment must satisfy interface naming, routing, and address-space requirements.

This section covers the following requirements for using Multus networking on an OpenShift Data Foundation deployment.

- General requirements
- Routing requirements
- Address space sizing requirements

7.4.1. General requirements

The general requirements for Multus networking are as follows:

- Interface used for the storage networks must have the same interface name on each OpenShift storage and worker node.
- Interfaces must all be connected to the same underlying network.
- When both public and cluster networks are used, they should not be connected to each other.
- At least one additional network for storage is required.
- Storage networks must be at least 10 Gbps. Bonded links are supported and suggested.

OpenShift Data Foundation with Multus creates many virtual MAC addresses on each storage network host interface. This requires network cards and switch hardware to support promiscuous mode. Some smart switches have security processes that inspect each packet, and these switches can bog down significantly due to the high number of MAC addresses for OpenShift Data Foundation. It is best to disable smart switch security features that require advanced processing.

Similarly, virtual or software switches might struggle to process the large number of MAC addresses. Some are not able to disable this functionality. Therefore, do not use virtual switches without a support exception.

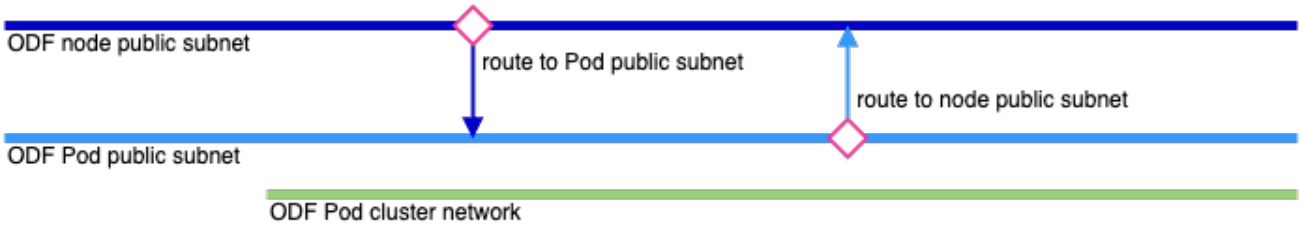
7.4.2. Routing requirements

For OpenShift Data Foundation Persistent Volume Claims (PVCs) to work properly, the Ceph-CSI driver pods deployed to each OpenShift node (workers and storage) must use host networking. Ceph-CSI drivers must also be able to reach OpenShift Data Foundation Ceph cluster through the Multus public network. This means that each OpenShift node must have a connection to the public network, and vice versa.

To make this as simple as possible without risk of IP overlap issues, select a different subnet (IP range) for node connections to the public network and Pod connections to the public network. Set up routing on the node public subnet and Pod public subnet such that they route to each other. The final result is that the node public subnet and Pod public subnet will act together as a single contiguous public network.

If the cluster network is used, do not route the Pod public network or node public network to the cluster network. The cluster network should be independent and isolated. The diagram below illustrates subnet routing requirements:

Figure 7.5. Subnet routing requirements



7.4.3. Address space sizing

OpenShift Data Foundation with Multus requires an IP address for each OpenShift Data Foundation Ceph storage daemon. Networks must have enough IP addresses to account for the number of storage pods that gets attached to the network, and some additional space to account for failover events. Expanding networks in the future might require downtime, so allocating space for the largest foreseeable cluster size is recommended.

For Multus public network, the following ranges must be planned:

- The Pod public network address space must include enough IPs for the total number of ODF pods running in the openshift-storage namespace
- The node public network address space must include enough IPs for the total number of OpenShift nodes (worker + storage)

For Multus cluster network, the following range must be planned:

- The Pod cluster network address space must include enough IPs for the total number of OSD pods running in the openshift-storage namespace

7.5. RECOMMENDATIONS FOR MULTUS NETWORKING

Following the recommended Multus networking practices ensures reliable storage performance.

This section describes the general and address range recommendations for Multus networking on OpenShift Data Foundation deployment.

7.5.1. General recommendations for Multus networking

Red Hat Ceph underpins OpenShift Data Foundation. Red Hat Ceph networking recommendations are useful for understanding more about OpenShift Data Foundation Multus networking and for finding specific recommendations. For more information, see link: Network considerations for Red Hat Ceph Storage that also applies to OpenShift Data Foundation networking.

7.5.2. Address ranges

The following table recommends IP ranges for each Multus network or subnetwork:

Table 7.1. Recommended IP ranges

Network	Network range CIDR	Approximate maximum	Public or cluster network
Pod public network	192.168.240.0/21	Total of 1600 OpenShift Data Foundation pods	Public network
Pod cluster network	192.168.248.0/22	800 OSDs	Cluster network
Public node network	192.168.252.0/23	400 OpenShift worker+storage nodes	Public network

This should suffice for most organizations. The recommendation uses the last 6.25% (1/16th) of the reserved private address space (192.168.0.0/16). Approximate maximum ODF and OpenShift sizing is noted, which accounts for 25% overhead to gracefully handle failure events.

7.6. PLANNING HOST INTERFACE CONNECTIONS

Planning host interface connections for Multus requires mapping each node type to the correct physical or VLAN interfaces.

OpenShift nodes can be separated into following three types:

- OpenShift control plane nodes
- OpenShift worker nodes
- OpenShift Data Foundation storage nodes.

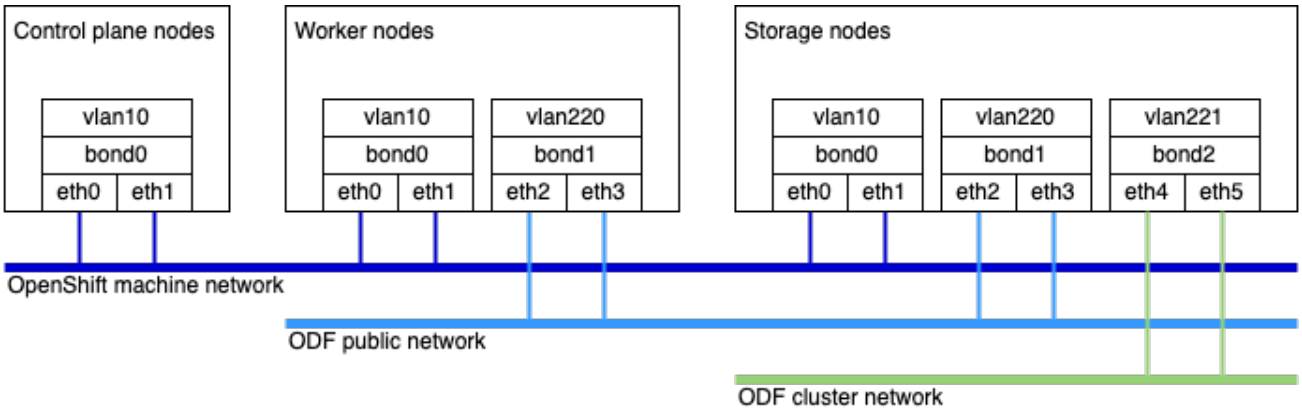
Each type requires different connections to the storage cluster as follows:

- All OpenShift nodes require access to the OpenShift machine network
- Control plane nodes do not require any connection to storage networks
  - If OpenShift Data Foundation PVCs are needed on control plane nodes (not recommended), treat them as worker nodes
  - If control plane nodes are used as OpenShift Data Foundation storage nodes, treat them as storage nodes
- Worker nodes run Ceph-CSI driver daemons to mount PVC storage to Pods and therefore require a connection to the OpenShift Data Foundation public network (if used)
- Storage nodes require access to all used storage networks

Example interfaces and connections

The following diagram shows example interfaces and connections that might be planned for each node type:

Figure 7.6. Example interfaces and connections



This example includes cluster network for completeness.

In this example, **vlan220** (public) and **vlan221** (cluster) are the interfaces to use for OpenShift Data Foundation configuration. If VLANs are unused in the deployed environment, **bond1** and **bond2** would be used instead. If no bonds or VLANs are used, host interfaces (for example, **eth2**, **eth4**) would be used instead. When using **Whereabouts** IPAM, IP addresses are not required on Multus parent interfaces such as NICs, bonds, or VLANs (for example, **eth2**–**eth4**, **bond1**–**bond2**, **vlan220**–**vlan221**).



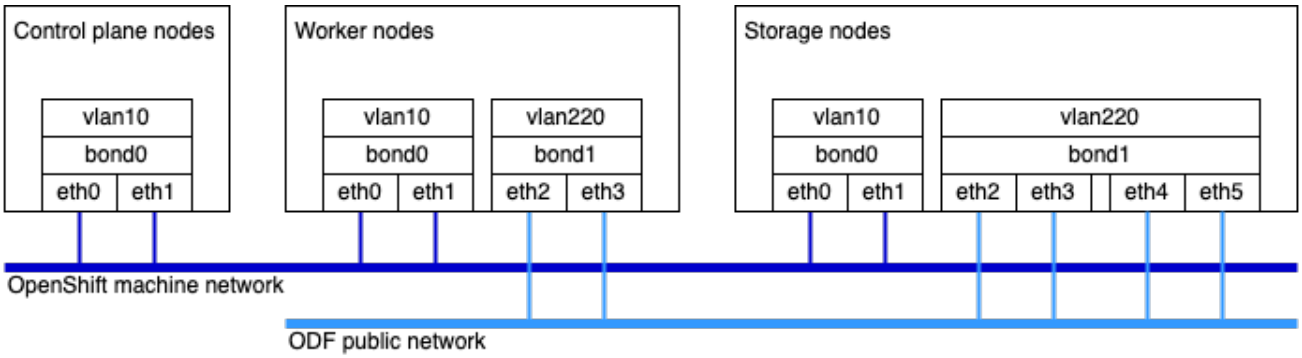
IMPORTANT

The “ODF public network” in the diagram accounts for both the OpenShift Data Foundation Pod public network and OpenShift Data Foundation node public network. The OpenShift machine network is used as a backbone for the OpenShift Pod network and is included for reference only. Do not mistake this network for the ODF node public network. Do not plan or execute routing that involves the machine network.

Recommended host interface connection

For an OpenShift Data Foundation Multus architecture using only the public network (recommended for most users), the same nodes from the example above can be used. **eth0**, **eth1**, **eth2**, and **eth4** would be bonded as **bond1** and tagged as **vlan220**. This architecture ensures that storage nodes have additional available bandwidth for replication and recovery needed by OpenShift Data Foundation Ceph OSDs. **vlan220** is be used for OpenShift Data Foundation configuration. However, many variations of valid environments exist.

Figure 7.7. Recommended host interface connection



There is no requirement to dedicate extra public network bandwidth to storage nodes, though this is beneficial. **eth4** and **eth5** can be omitted from storage nodes entirely to dedicate the same bandwidth to both worker and storage nodes. This design has been commonly seen in customer environments.

Alternatively, **eth2** and **eth3** can be higher-throughput interfaces on storage nodes (for example, 10Gbps on worker nodes and 25Gbps on storage nodes). This allows for more storage node bandwidth while still using only two interfaces on storage nodes.

7.7. MULTUS NETWORK CONFIGURATION

Multus networks are configured with **NetworkAttachmentDefinition** objects. These objects configure Pod network connections.

OpenShift Network Attachment Definitions support many options, however, OpenShift Data Foundation supports only a few that are outlined as follows:

- **macvlan** is the only supported CNI plugin **whereabouts** is the only supported IP Address Management (IPAM) plugin

Diverging from these selections requires a support exception

Address range values must also match those selected for the cluster. The example cluster, example host interfaces, and recommended network ranges are used here. Modify them as needed in the deployed environment.

You can configure your Multus networks to use IPv4 or IPv6. Multus networks can only be configured to use either IPv4 or IPv6. Mixing modes is not supported.

7.7.1. Public NetworkAttachmentDefinition

The recommended Pod public network configuration for the example is as follow. Omit this if the Multus public network is not used. Make sure to include the **routes** section that allows Pods on the public network to reach nodes through the public network.

```
apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: public-net
  namespace: openshift-storage
spec:
  config: |
    {
      "cniVersion": "0.3.1",
      "type": "macvlan",
      "master": "vlan220", # host public network interface
      "mode": "bridge",
      "ipam": {
        "type": "whereabouts",
        "range": "192.168.240.0/21", # pod public network range
        "routes": [
          {"dst": "192.168.252.0/23"} # node public network range
        ]
      }
    }
```

7.7.2. Cluster NetworkAttachmentDefinition

The recommended Pod cluster network configuration for the example is as follows. Omit this if the Multus cluster network is not used (recommended for most users). This configuration should not have a **routes** section.

```
apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: cluster-net
  namespace: openshift-storage
spec:
  config: |
    {
      "cniVersion": "0.3.1",
      "type": "macvlan",
      "master": "vlan221", # host cluster network interface
      "mode": "bridge",
      "ipam": {
        "type": "whereabouts",
        "range": "192.168.248.0/22" # pod cluster network range
      }
    }
```

7.7.3. Node configuration for Multus public network

OpenShift worker and storage nodes must be configured to route host traffic to the Pods on the public network through the host public network interface.

The recommended way to configure nodes is using OpenShift **NodeNetworkConfigurationPolicy** objects. This method can be supported for all OpenShift users regardless of deployment strategy. This method requires the NMState Operator to be installed and enabled. For more information, see link: Kubernetes NMState Operator.

Each node must obtain an IP address on the ODF public network in the node public network address range. Static IP address management is the only IPAM method that can be supported for any OpenShift cluster. Thus, static management is OpenShift Data Foundation supports only the static management method. This requires one **NodeNetworkConfigurationPolicy** object per host. The template that can be used to configure a host is shown below.



IMPORTANT

This template below creates an interface called “shim” interface on each host. The shim interface uses the host public network interface (for example, **vlan220**) as a parent. The static IP is then given to the shim interface and not to the parent. Similarly, routing uses the shim. This is a critical detail. Macvlan disallows the virtual network of connected Pods on any given host from reaching the host directly or through switch hairpin. Without the shim interface, OpenShift Data Foundation will not function properly. Do not attempt to set up the OpenShift Data Foundation Multus public network without configuring the shim interface.

```
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: ceph-public-net-shim-{{NODE_NAME}}
spec:
  nodeSelector:
    node-role.kubernetes.io/worker: ""
    kubernetes.io/hostname: {{NODE_NAME}}
  desiredState:
    interfaces:
      - name: odf-pub-shim
        description: Shim interface to connect to ODF public network
        type: mac-vlan
        state: up
        mac-vlan:
          base-iface: vlan220 # host public network interface
          mode: bridge
          promiscuous: true
        ipv4:
          enabled: true
          dhcp: false
          address:
            - ip: 192.168.252.1 # static IP in node public network range
              prefix-length: 23 # node public network range mask
    routes:
      config:
        - destination: 192.168.240.0/21 # pod public network range
          next-hop-interface: odf-pub-shim
```

First, follow comments in the template to update the base template for the environment being deployed. Then, for each node, copy the template, and fill in **{{NODE_NAME}}** and a unique static IP for each node.

7.8. PREINSTALLATION CHECKLIST FOR MULTUS CONFIGURATION

Completing the preinstallation checklist for Multus before deploying OpenShift Data Foundation prevents issues after the storage cluster is running.

After all the previous sections have been planned or executed, OpenShift Data Foundation preinstallation can begin. The following checklist of preinstallation items should be completed before proceeding further:

1. Read the Multus architecture document in its entirety.
2. Select which OpenShift Data Foundation Multus networks to use.
3. Plan host interface connections.
4. Deploy OpenShift.
5. Select network address ranges to use.
6. Write **NetworkAttachmentDefinition** manifests for Pods.
7. Write **NodeNetworkConfigurationPolicy** manifests for nodes.
8. Deploy the manifests to the OpenShift cluster
9. Install OpenShift Data Foundation, but do not deploy a StorageCluster yet
10. Run the Multus validation tool to check preinstallation deployments (covered in the next section)

7.9. VALIDATING AND DEBUGGING MULTUS CONFIGURATION

OpenShift Data Foundation has a tool to perform basic connectivity tests prior to OpenShift Data Foundation StorageCluster deployment. This Multus validation tool is intended to be a self-help tool that returns a list of details that might be incorrect based on the validation steps that pass or fail.

This tool performs basic connection and IP availability checking. It cannot detect every possible misconfiguration, and it cannot detect network performance issues. Ensure due diligence for each cluster deployment.

For more information, see link: Appendix A. Multus prerequisites validation tool.

When encountering an issue with the tool, check the tool output for suggestions. Use events associated with Pods that the tool runs (for example, using **oc -n openshift-storage describe pods**) to debug further. If issues persist despite debugging, collect OpenShift Data Foundation must-gather, and open an OpenShift Data Foundation help ticket that includes full tool output and the must-gather tarball. For more information, see link: Downloading log files and diagnostic information using must-gather .

For best results, use a configuration file with the Multus validation tool. The following configuration is recommended, which can be updated by following the comments as needed.

```
publicNetwork: "public-net" # comment out if public network is unused
```

```
# clusterNetwork: "cluster-net" # uncomment if used


# for airgapped environments, mirror this image, then update this config to reference the mirror
nginxImage: "quay.io/nginx/nginx-unprivileged:stable-alpine"

namespace: "openshift-storage"
serviceAccountName: "rook-ceph-system"

nodeTypes:
  storage-nodes:
    osdsPerNode: 12 # set to the max number of OSDs per storage node
    otherDaemonsPerNode: 16
    placement:
      nodeSelector:
        cluster.ocs.openshift.io/openshift-storage: ""
      tolerations:
        - {"key":"node.ocs.openshift.io/storage","value":"true"}
  worker-nodes:
    osdsPerNode: 0
    otherDaemonsPerNode: 6
    placement:
      nodeSelector:
      tolerations:
```

If this recommendation proves to be inadequate or out of date, use the tool to generate a new configuration using **odf multus validation config dedicated-storage-nodes**.

7.10. OPENSHIFT DATA FOUNDATION INSTALLATION AFTER VALIDATING MULTUS CONFIGURATION AND POST INSTALL CHECKS

After validating the connectivity using the Multus validation tool, OpenShift Data Foundation storage cluster can be deployed. Refer to OpenShift Data Foundation documentation and make sure to select the appropriate Multus networks during deployment.

Post-installation checks

In addition to the OpenShift Data Foundation health checking, ensure that Multus is configured as expected by performing the following:

- Using the ODF toolbox or **odf-cli** tool, run **ceph osd dump** to verify that OSDs have an IP in the cluster network.
  In the output, ensure that OSD daemons have an IP in the public network (if used). OSDs should additionally have an IP in the cluster network, if the Multus cluster network was used. If it was left empty, OSDs should only list public network IPs. If a filesystem has been deployed, ensure that MDS daemons have an IP in the public network (if used).

- Create a test application Deployment in a new namespace that uses ODF PVCs.
  Ensure that the test application Pod can write to the mounted PVC volumes. Delete the Deployment's Pod, then wait for it to reschedule to a new node. Ensure that the previous data can be read back, and ensure new data can be written.

- Run some test load generators in the cluster to write to OpenShift Data Foundation PVCs. After the OpenShift Data Foundation cluster has filled to 20%, induce a hard failure of a node or failure zone. Leave the node or zone in failure state for an hour, and observe the OpenShift

Data Foundation cluster status and OpenShift Data Foundation Pod statuses. Cluster health can be expected to show **HEALTH_WARN**, but it should not show **HEALTH_ERR**. One or two OpenShift Data Foundationd Pod failures may be okay, but several failures could indicate a problem.

7.11. ENABLING MULTUS SUPPORT ON AN EXISTING OPENSHIFT DATA FOUNDATION CLUSTER

You can add Multus networking to an existing OpenShift Data Foundation cluster.

This procedure describes how to configure Multus networking for an existing OpenShift Data Foundation deployment.

Prerequisite

- Before proceeding, review Multus networking requirements and Multus network configuration. Ensure that the environment meets these prerequisites before migrating the cluster from a non-Multus network to a Multus-enabled network.

Procedure

1. Configure the nodes.
   As described in the architecture documentation, the recommended method for configuring node-level network settings is to use the **NodeNetworkConfigurationPolicy** (NNCP).

   Using NNCP requires the NMState Operator to be installed and enabled on the cluster. NNCPs are required only when a **public network** will be used.

   Example NodeNetworkConfigurationPolicy for node compute-0

```
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  annotations:
    nmstate.io/webhook-mutating-timestamp: "1743035283553699179"
  name: ceph-public-net-shim-compute-0
spec:
  desiredState:
    interfaces:
    - description: Shim interface used to connect host to OpenShift Data Foundation public
Multus network
      ipv4:
        address:
        - ip: 192.168.252.1
          prefix-length: 24
        dhcp: false
        enabled: true
      mac-vlan:
        base-iface: ens224
        mode: bridge
        promiscuous: true
      name: odf-pub-shim
      state: up
      type: mac-vlan
```

```
    nodeSelector:
      kubernetes.io/hostname: compute-0
      node-role.kubernetes.io/worker: ""
```

2. Create NetworkAttachmentDefinition (NAD) objects
   Before creating NADs, review the NAD requirements and the NAD configuration guidance provided in the Multus documentation.

   The following examples show both public and cluster network NADs. Only the NADs required for your deployment need to be created.

   Example public network NAD

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: public-net
  namespace: default
spec:
  config: '{"cniVersion": "0.3.1", "type": "macvlan", "master": "ens224", "mode":
    "bridge", "ipam": {"type": "whereabouts", "range": "192.168.20.0/24", "routes":
    [{"dst": "192.168.252.0/24"}]}}'
```

   Example cluster network NAD

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: cluster-net
  namespace: default
spec:
  config: '{"cniVersion": "0.3.1", "type": "macvlan", "master": "ens224", "mode":
    "bridge", "ipam": {"type": "whereabouts", "range": "192.168.30.0/24"}}'
```

3. Run the Multus validation tool.
   Run the validation tool as described in the Multus validation documentation. A successful validation produces output similar to the following:

```
RESULT: multus validation test succeeded
```

   This result confirms that the network configuration is valid.

4. Edit the StorageCluster custom resource.
   After the validation tool succeeds, update the **StorageCluster** CR to configure Multus networking. Modify the **network** section to include the required providers and selectors.

   Example StorageCluster networking configuration

```
network:
  multiClusterService: {}
  provider: multus
  selectors:
    cluster: default/cluster-net
    public: default/public-net
```

This example includes both cluster and public networks. Specify only the networks required for your deployment.

5. Move MDS pod to Multus network manually as it does not get moved automatically.
Restart the MDS pods after the OSD pods are moved to Multus network by running the following command:

```
num_osds="$(oc --namespace openshift-storage get pod -l app=rook-ceph-osd --no-headers
| wc -l)"

while true; do
  osds_with_multus="$(oc --namespace openshift-storage get pod -l app=rook-ceph-osd |
grep -c 'k8s.v1.cni.cncf.io/networks')"
  [[ $osds_with_multus -eq $num_osds ]] && break
  sleep 10
done

while true; do
  osds_running="$(oc --namespace openshift-storage get pod -l app=rook-ceph-osd | grep -c
'Running')"
  [[ $osds_running -eq $num_osds ]] && break
  sleep 10
done

oc --namespace openshift-storage delete pod -l app=rook-ceph-mds
```

This gets the OSD pod count. Then it checks OSD pods with Multus network and confirms that the OSD pods are running. Later it deletes the MDS pods.

6. Validate the Ceph network configuration.
After updating the StorageCluster, verify that Ceph OSDs are using the Multus network addresses.

Run the following command:

```
$ ceph osd dump | grep osd
```

Example output:

```
osd.0 ... [v2:192.168.20.8:6800/... ,v1:192.168.20.8:6801/...]
[v2:192.168.30.1:6800/...,v1:192.168.30.1:6801/...]
osd.1 ... [v2:192.168.20.9:6800/... ,v1:192.168.20.9:6801/...]
[v2:192.168.30.2:6800/...,v1:192.168.30.2:6801/...]
osd.2 ... [v2:192.168.20.10:6800/... ,v1:192.168.20.10:6801/...]
[v2:192.168.30.3:6800/...,v1:192.168.30.3:6801/...]
```

In this example:

- The public network range is: **192.168.20.0/24**
- The cluster network range is: **192.168.30.0/24**
The OSDs show addresses in both ranges, confirming successful Multus integration.

APPENDIX A. MULTUS PREREQUISITE VALIDATION

The multus validation command verifies that the Multus and system configuration is compatible with Rook. It is suggested to run it before installing Rook to ensure network compatibility.

The validation test starts up a web server and many clients to verify that Multus network communication works properly. It is a long-running test.

It does not perform any load testing. Networks that cannot support high volumes of Ceph traffic might still encounter runtime issues. This might be noticeable with high I/O load or during OSD rebalancing (for example, during node/disk failure or during OpenShift Data Foundation upgrade). Therefore, it is recommended to perform network load testing to ensure that base network configurations meet user requirements. To know more about OSD rebalancing refer to the documentation.

Subcommands

- **config**: Generate a sample validation config file
- **run**: Run Multus validation tests to verify network connectivity
- **cleanup**: Clean up resources from a previous validation run

Procedure

You can use config file, which is the preferred way before running the validation tool. Generate a sample, edit it for the deployment, then run validation with it:

```
$ odf multus validation config dedicated-storage-nodes > multus-validation-config.yaml
```

Edit **multus-validation-config.yaml** as needed, for example: set **publicNetwork** and **clusterNetwork** to the Multus network attachment names, adjust namespace, node placement, and so on. Then run:

```
$ odf multus validation run --config multus-validation-config.yaml
```

Running with flags (Not Recommended)

To run without a config file, pass all options on the command line. Default namespace is **openshift-storage**:

```
$ odf multus validation run \
  --namespace openshift-storage \
  --public-network [namespace]/public-net \
  --cluster-network [namespace]/cluster-net
```

For all options:

```
$ odf multus validation run --help
```

Cleanup

To remove leftover resources from a previous validation run:

```
$ odf multus validation cleanup --help
```

A.1. TROUBLESHOOTING VALIDATION TOOL

If the test fails when you run the validation tool, it suggests some things to check based on the failure condition. This helps to resolve common configuration issues quickly.

- For diagnosing issues with the tool itself, use the --log-level DEBUG for getting additional details.
- If pods are not starting, you can expect the following issues:
  - Problem with NAD configuration.
  - The **oc pod describe** command contains the error as an event.
  - Some NICs do not support enough virtual ports or VLANs for the additional MAC addresses.
- If pods are starting but not communicating, you can expect the following issues:
  - Network design or configuration might contain errors.
  - Network switch might be blocking sub-interface MAC addresses or IPs (check promiscuous mode settings)
  - Switch firewall
  - System or NAD Linux networking configurations might be blocking the traffic
  - SOS report will have good information to look at next
    - **ip_netns_exec_*_address_*** and **ip_netns_exec_*_route_*** from container namespaces

A.2. HOW TO GET HELP

If you are having trouble resolving the issue based on these troubleshooting steps described in the previous section, a substantial amount of information should be collected for getting help.

The tool will list resources that should be collected into an archive file in this case. For OpenShift and OpenShift Data Foundation, the following OpenShift tools contains the information needed for further debugging:

- Network diagrams
- ODF must-gather
- OCP must-gather
- OCP must-gather network logs
- SOS reports from all nodes on which test pods are running (https://access.redhat.com/solutions/3530881)

Multus is highly integrated among various product layers, and issues are most likely to be related to configuration or the physical hardware environment rather than product bugs. Initial investigation at the engineering layer will involve ODF, OpenShift, and possibly RHEL networking experts to rule out configuration or environment.

- General Multus bugs/help: OCP Jira: https://issues.redhat.com/projects/OCPBUGSM
- ODF Multus bugs: ODF jira: https://issues.redhat.com/projects/DFBUGS
- Contact Red Hat customer Support.